universität
innsbruck

Final presentation: Segfault Slayer

Schmölz Lukas, Schöpf Emanuel, Schweigl Dominik

Used Tools

Build Tools

- SFML
- Tileson + Tiled
- Catch2

Asset Sources

- Pixel art: Pixellab, Gemini
- Audio: Pixabay (Sounds), itch.io (Music)

Game Overview

Game Idea

- Metroidvania-style 2D side-scroller
- Player fights programming concepts and bugs
- Playful, Pixel-theme

World Design

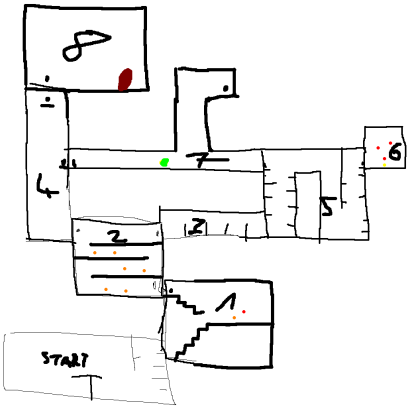
- First Area: Hardware with electric components roaming around
- Second Area: Abstract programming concepts escaped from the bosses lab

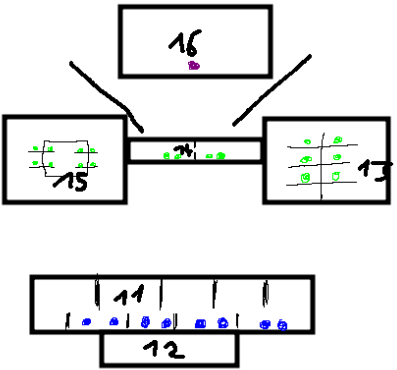
Team Responsibilities

- Emanuel
  - (1) map
  - (1) save / load with dedicated save points (rooms)
  - (2) two consecutive areas
- Lukas
  - (2) enemies
  - (1) menus
- Dominik
  - (2) basic movement
  - (1) one or more advanced movement mechanics
  - (1) main combat

Minimaps

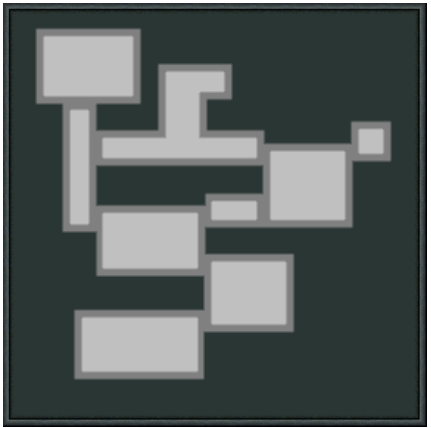
Initial Scribbles:



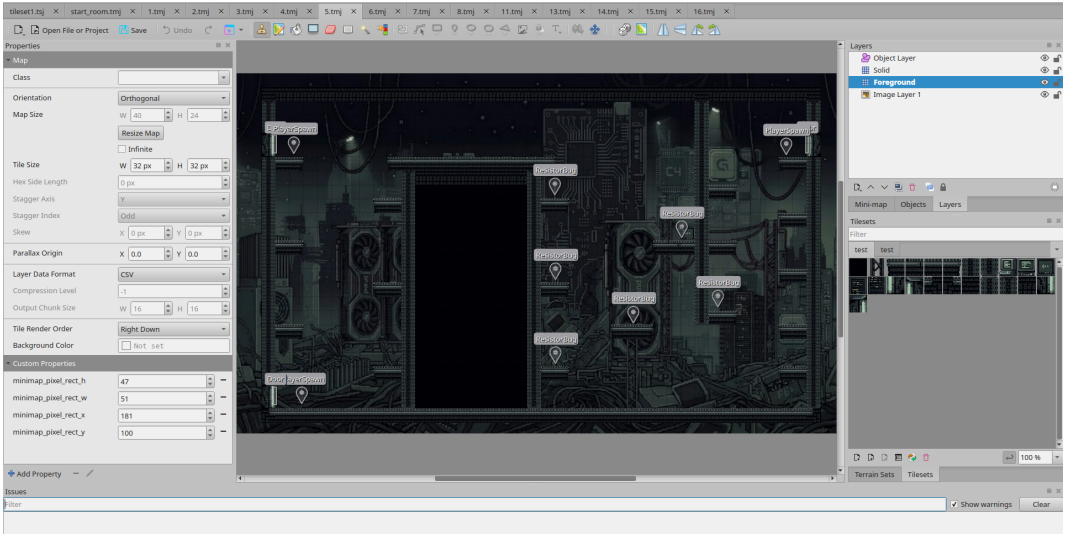


Minimaps

current:
(work in progress)



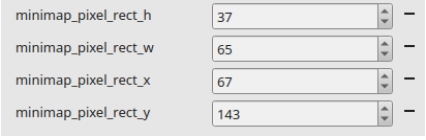
Our Tiled Setup

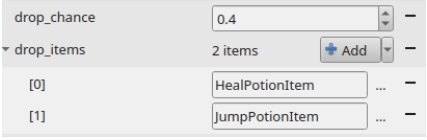


Data Design

Flexibility & Extensibility
some examples:

- Loot Drops
- Door Locked
- Room Minimap FloatRect





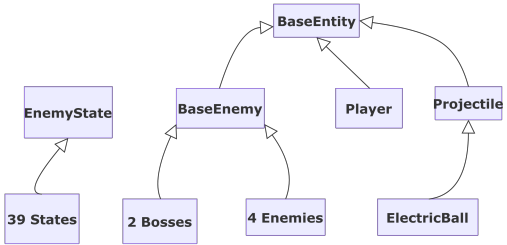
Saving and Loading World

```cpp
json j;
try {
    j["player"] = player.serialize();

    j["rooms"] = json::array();
    for (const auto &room : rooms) {
        json j_room = room.second.serialize();
        j_room["id"] = room.first;
        j["rooms"].push_back(j_room);
    }

    j["currentRoom"] = getCurrentRoomId();

    std::ofstream file("saves/" + worldName + ".json");
    if (!file.is_open()) {
        std::cerr << "Failed to open save file\n";
        return;
    }

    file << j.dump(4);
} catch (const std::exception &e) {
    std::cerr << "Serialization error: " << e.what() << "\n";
}
```

```cpp
try {
    if (j.contains("player")) {
        player.deserialize(j["player"]);
    }

    if (j.contains("rooms")) {
        for (const auto &j_room : j["rooms"]) {
            std::string id = "";
            if (j_room.contains("id"))
                id = j_room["id"];

            if (rooms.find(id) == rooms.end()) {
                std::cerr << "Unknown room id in save: " << id << "\n";
                continue;
            }

            Room &room = rooms[id];
            room.deserialize(j_room);
        }
    }

    if (j.contains("currentRoom")) {
        setCurrentRoom(j["currentRoom"]);
    }

} catch (const std::exception &e) {
    std::cerr << "World loading error: " << e.what() << "\n";
}
```

Serialization

Peter Thoman: Handle serialization continuously instead of implementing everything at once at the end.




```
class BaseEntity {
  public:
    virtual json serialize() const;
    virtual void deserialize(const json &j);
```

Factory Pattern

```cpp
using json = nlohmann::json;

struct EnemyFactory {
    static std::unique_ptr<BaseEnemy> create(const json &j)
    {
        if (j.empty())
            return nullptr;
        const std::string type = j["type"];

        if (type == "RaceConditionSlime") {
            auto enemy = std::make_unique<RaceConditionSlime>(sf::Vector2f{0.f, 0.f}, BaseEnemy::DROP_CHANCE);
            enemy->deserialize(j);
            return enemy;
        }

        if (type == "Capacitor") {
            auto enemy = std::make_unique<Capacitor>(sf::Vector2f{0.f, 0.f}, BaseEnemy::DROP_CHANCE);
            enemy->deserialize(j);
            return enemy;
        }
```

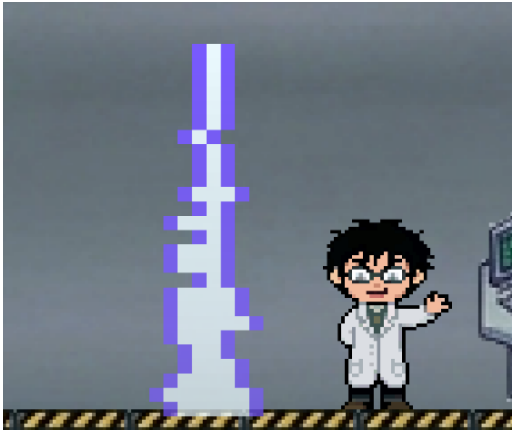
Demo & Code Review

Boss Design

Transistor

- Roams the boss roam and shoots projectiles
- Shocks area around him when player too close
- Spawns additional minions in phase 2

Boss Design

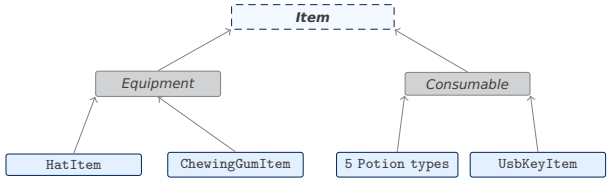


Segfault

- Spawns area attacks and environment blockers
- Summons minions in phase 2
- Glitches into 2 versions of himself in phase 3

Items

- **Equipment** – wearable items occupying a fixed slot; `activate()` moves to `equipmentSlot()` returning the target `SlotKind`
- **Consumable** – single-use items; `activate()` applies the effect and removes the item from inventory



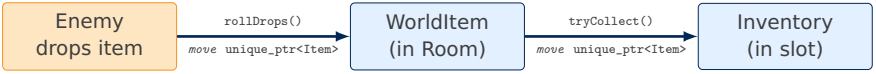
Equipment

Hat Gum Backup

Consumables

Heal Speed Jump Damage Resist USB Key

WorldItem & Inventory: Ownership Chain



Transfer 1 – drop (game_scene.cpp)

```
for (auto& enemy :
        room->enemies_) {
  if (!enemy->isAlive()) {
    for (auto& drop :
            enemy->rollDrops()) {
      auto wi = make_unique<
        WorldItem>(
          enemy->getPosition(),
          std::move(drop));
      room->appendItem(wi);
    }
  }
}
```

Transfer 2 – collection (game_scene.cpp)

```
auto collected =
  worldItem->tryCollect(
    player_.getBounds());
if (collected)
  player_.inventory()
    .addItem(
      std::move(collected));
// Room erases next frame:
items_.erase(
  remove_if(
    items_.begin(),
    items_.end(),
    [](const auto& i){
      return i->isCollected();
    }),
  items_.end());
```

Special case: BackupDisk

```
// checked every frame
if (!player_.isAlive()
    && player_.inventory()
         .hasBackup()) {
  player_.heal(1);
  player_.inventory()
    .clearSlot(
      {SlotKind::Backup, 0});
  player_.triggerIframes();
}
```

Consumed *automatically* – not through `interact()` but directly by the scene.

Inventory System: Technical Design

Uniform Slot Addressing

```
enum class SlotKind {
    Hat, Gum, Backup, Grid, Hotbar
};
struct SlotRef { SlotKind kind; int index; };
```

Every slot – equipment, grid, or hotbar – is a single `SlotRef`. `moveItem(from, to)` and `interact(slot, player)` are generic entry points regardless of which area is involved.

Slot-Move Validation

```
bool isValidInSlot(const Item& item, SlotRef s) {
    auto home = item.equipmentSlot();
    if (s.kind == Hat || s.kind == Gum
                      || s.kind == Backup)
        return home.has_value()
            && home.value() == s.kind;
    return true; // grid + hotbar: any item
}
```

Validation lives in the model – UI drag-drop and keyboard shortcuts obey the same rules.

Item Self-Activation

```
// Equipment (e.g. HatItem)
void activate(...) {
    inventory.moveToEquipmentSlot(ownSlot);
}
// Consumable (e.g. HealingPotionItem)
void activate(...) {
    player.heal(amount);
    inventory.clearSlot(ownSlot);
}
```

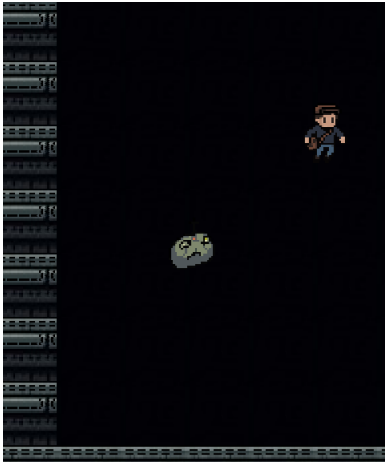
The inventory calls `item.activate(player, *this, slot)`. The item decides what to do – the inventory is a passive store; **logic lives in the item**.

UI Interactions

- **Drag-drop**: validates via `isValidInSlot` before completing move
- **Shift+click**: quick-swaps grid ↔ hotbar
- **Keys 1–7**: assign hovered item to hotbar slot

Final Sprint

- Last Music and Audio wiring
- Item & enemy placement improvement
- Second Area rework
- Story snippets
- Refactoring



Lessons Learned

- Focus on creating a MVP early
- Creative Work just as important as gameplay mechanics
- Better Milestone / Issue Management